

Introduction

The purpose of the product is to simulate drones flying in a lattice cube formation in 3D using Unity. The main features include rendering drones with arrows pointing. The main features include rendering drones, arrows, and cubes. The drones fly in a specific pattern delineated by the arrows. The transparent cubes group together 8 drones at a time in a chess grid pattern to indicate the alternating drone formation in each lattice cube. The cubes can be toggled on and off using the on-screen UI. Two different perspectives are provided for users to understand the simulation. The automatic camera circles the drones on each iteration, while the drone perspective follows the view of one drone throughout the simulation. The target users of this simulation are researchers, students, and engineers interested in visualizing path-planning algorithms and decision-making behaviors in localized swarm robotics systems.

All use cases are executed independently each frame, with no memory of previous frames. Most view-layer components rely on abstract functions provided by the Unity Engine. UCs 1, 6, and 7 are always simultaneously executing. Depending on the user's configuration, either UCs 2, 3, and 4 or UC 5 will be selected to execute alongside UCs 1, 6, and 7. If the user configures the simulation to run UCs 2, 3, and 4, each frame will select one use case to execute based on the data files.

Use Cases

Use Case 1: Render drones with arrows pointing to their respective final destination

Precondition: user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed
2. Read the current Drone Movement File in need
3. Update charging drone with its latest position by using interpolation between the spawn position and entry position into the formation.
4. Generate/Update drones with their latest position by using interpolation between the start/end position provided in the file.
5. The system renders the drones onto the screen
6. and correspond each drone with an arrow pointing to the destination

Alternate Flows:

Flow#1

1a: The new frame is the first frame after the precondition (in Unity abstract function Start() can handle the specific scenario)

2: The system additionally renders a charging block somewhere beneath the drones

3: The system additionally renders a charging drone above the charging block

4: Resume at Main Flow 4.

Flow#2

3a: The latest position is at the end position of the provided file.

4: Read the next available Drone Movement File

5. Render an arrow that starts from the latest position and points toward the new end position from the new file for each drone

6. Rotate the drones to face the arrows direction if the rotation is in the horizontal plane

7. Resume at Main Flow 5.

Use Case 2: Run Mode Automatic camera

Precondition: user selects toggle AutoCam to true then user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed

2. Read the Drone Movement File drone mode field as “run”
3. Stop drone movement
4. Update camera position based on algorithm tracking how to look at the simulation
5. The system renders the result onto the screen

Flow#1

4a: If according to algorithm, the camera position no longer needs change.

2: The system stops camera movement

3: The system resumes drone movement in use case 1.

4: Resume at Main Flow 5.

Use Case 3: Add Mode Automatic camera

Precondition: user selects toggle AutoCam to true then user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed
2. Read the Drone Movement File drone mode field as “fill”
3. Stop drone movement
4. Update camera position based on algorithm tracking how to look at the simulation
5. The system renders the result onto the screen

Flow#1

4a: If according to algorithm, the camera position no longer needs change.

2: The system stops camera movement

3: The system resumes drone movement in use case 1.

4: Resume at Main Flow 5.

Use Case 4: Subtract Mode Automatic camera

Precondition: user selects toggle AutoCam to true then user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed
2. Read the Drone Movement File drone mode field as “secondary” after the previous mode field as “file”
3. Update camera position based on algorithm tracking how to look at the simulation
4. The system renders the result onto the screen

Use Case 5: Drone Perspective Camera

Precondition: user unselect toggle AutoCam to false then user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed
2. The camera follows the current drone attached and has identical facing direction
3. The system renders the result onto the screen

Flow#1

2a: If the current following drone no longer exists.

2: The system designates the camera to follow the new drone at the position where the original drone disappeared.

4: Resume at Main Flow 3.

Flow#1

1a: The new frame is the first frame after the precondition (in Unity abstract function Start() can handle the specific scenario)

2: The system additionally renders a four-sided wall.

3: Resume at Main Flow 3.

Use Case 6: Transparent Cube Rendering

Precondition: user hits play button on the Unity screen.

Main Flow:

1. A new frame is being executed
2. Read the current order in Movement Generation File
3. The user toggles on/off the cube visualization from the screen
4. Transparent cubes appear/disappear wrapping each eight drones group that forms the eight corners of a cube with chess pattern alternating color
5. The system renders the results on the screen

Flow#1

2a: If the current order is a new order with add/sub compared to last time when this use case is executed.

3: New cube is added at the position specified in file for add. Cube gets deleted at the position specified in file for sub.

3: Resume at Main Flow 3.

Use Case 7: Adjustable Light

Precondition: user hits play button on the Unity screen.

Main Flow:

6. The user adjusts the light bar on the screen
7. The UI shows the light intensity value change
8. The system adjust the lighting of the scene to the desired value
9. The system renders the result onto the screen

Notes for Algorithms:

According to implementation of algorithms for use case 2 to 4, it is known that they require information of current Drone Movement File, current order in the Movement Generation File. All three use cases are guaranteed to have different algorithms to fulfill the requirement.

Data Files

Currently, there are one group of files and another individual file in use.

There may be further improvement of storing these data in database rather than tons of CSV files. In the near future, no adjustment is foreseen.

Drone Movement Files

The one group of data files provides the series of start/end position for all drones existing in the simulation. Each file within the group has all the drone ID, start position, end position, color, and drone mode.

Movement Generation File

The other individual file provides a sequence of orders to generate the group of files mentioned. The individual file has the format (generation mode, x, y, z, subtract axis). Generation mode has three modes: add, sub, run. Add/Sub indicates if adding/subtracting a lattice cube indicated at x, y, z position. Run indicates looping the current structure with x amount of time. Subtract axis only has a value in sub generation mode. It indicates with this lattice cube gone, it slices off one full lattice cube width of the plane orthogonal to the axis direction. (e.g. -x means subtracting the yz plane that sits at position x given in data.